Zudumradošā saspiešana un kļūdu korekcija: Beigu uzdevumu atrisinājumi

Uzdevumu krājuma [Zudumradošā saspiešana un kļūdu korekcija: Beigu uzdevumi]({{ '/lectures/lossy_final_problemsets/' | relative_url }}) atrisinājumi. Apzīmējumi ir tādi paši kā lekcijās. Datoruzdevumiem dotas īsas Python programmas un rezultāti, kas iegūti ar Python 3.14, NumPy, SciPy 1.17, Pillow 12.1 un `galois` 0.4.8. Citās bibliotēku versijās un ar citu gadījuma skaitļu sēklu skaitļi var nedaudz atšķirties, bet secinājumi paliek tie paši.

JPEG saspiešana

1. uzdevums (DCT ar roku)

(a) No matricas C rindām:

$$\begin{aligned} y_0 &= a\,(4+8+8+4) = 12, \\ y_1 &= b\,(4-4) + c\,(8-8) = 0, \\ y_2 &= a\,(4-8-8+4) = -4, \\ y_3 &= c\,(4-4) - b\,(8-8) = 0. \end{aligned}$$

Tātad $y = (12, 0, -4, 0)$. Parsevāla vienādība: $\sum x_n^2 = 16+64+64+16 = 160$ un $\sum y_k^2 = 144+16 = 160$.

(b) Bāzes vektors $c_k[n] = \alpha_k \cos\frac{\pi(2n+1)k}{2N}$ apmierina $c_k[N-1-n] = \alpha_k \cos\left(\pi k - \frac{\pi(2n+1)k}{2N}\right) = (-1)^k c_k[n]$. Tātad pāra k bāzes vektori ir simetriski, bet nepāra k – antisimetriski (tas redzams arī matricā C: $(b, c, -c, -b)$ un $(c, -b, b, -c)$).

Apgalvojums: ja $x_n = x_{N-1-n}$ visiem n, tad $y_k = 0$ visiem nepāra k.

Pierādījums: $y_k = \sum_n x_n c_k[n]$. Sagrupējam saskaitāmos pāros n un $N{-}1{-}n$: $x_n c_k[n] + x_{N-1-n} c_k[N{-}1{-}n] = x_n(c_k[n] - c_k[n]) = 0$, ja k ir nepāra (ja N ir nepāra, vidējam elementam $c_k[\frac{N-1}{2}] = -c_k[\frac{N-1}{2}] = 0$). $\square$

(c) $12/10 = 1.2 \to 1$, $-4/10 = -0.4 \to 0$, tātad $\hat{y} = (10, 0, 0, 0)$ un $x' = C^T\hat{y} = 10 \cdot (a, a, a, a) = (5, 5, 5, 5)$. Kvadrātiskā kļūda $\sum(x_n - x'_n)^2 = 1+9+9+1 = 20$, un $\sum(y_k - \hat{y}_k)^2 = 2^2 + 4^2 = 20$ – sakrīt, jo $x - x' = C^T(y - \hat{y})$ un C ir ortonormēta. Blokā paliek tikai vidējā vērtība (turklāt nedaudz par mazu: 5, nevis 6).

(d) Katra kolonna ir konstanta (v, v, v, v), un tās DCT ir $(2v, 0, 0, 0)$ (jo $y_0 = a \cdot 4v = 2v$, bet pārējās rindas C summa ir 0). Tātad CA ir matrica, kuras pirmā rinda ir $2 \cdot (4, 8, 8, 4) = (8, 16, 16, 8)$, bet pārējās rindas – nulles. Reizinot ar C^T no labās, transformējam rindas: nulles rindas paliek nulles, bet pirmās rindas DCT ir $2y = (24, 0, -8, 0)$:

$$B = \begin{pmatrix} 24 & 0 & -8 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Blokam ir tikai horizontālas frekvences (visas rindas vienādas), tāpēc nenulles ir tikai pirmā rinda.

(e) Atņemot konstanti 128, blokam A pieskaita $-128 \cdot \mathbf{1}$, kur $\mathbf{1}$ ir vieninieku matrica. Tā ir DC bāzes attēla $c_0 c_0^T$ daudzkārtnis ($c_0 c_0^T = \frac{1}{8}\mathbf{1}$), tāpēc mainās tikai $B_{0,0}$, pārējie koeficienti ir ortogonāli konstantei. $B_{0,0} = \sum_{x,y} A_{x,y} \cdot \frac{1}{8} = 8\bar{A}$, tātad $B_{0,0}$ samazinās par $8 \cdot 128 = 1024$. Bez nobīdes $B_{0,0} \in [0; 2040]$, ar nobīdi – $[-1024; 1016]$; DC vērtības ir mazākas un simetriskas ap nulli. (Tas pats bija 1. uzdevuma (d) punktā: $B_{0,0} = 24 = 4 \cdot 6$, kur 6 ir bloka vidējā vērtība un $4 = N$.)

2. uzdevums (Krāsu telpas)

(a)

$$\begin{aligned} Y &= 0.299 \cdot 255 + 0.587 \cdot 128 + 0.114 \cdot 0 = 76.245 + 75.136 = 151.381 \to 151, \\ \mathrm{Cb} &= 128 + \tfrac{0-151.381}{1.772} = 128 - 85.43 = 42.57 \to 43, \\ \mathrm{Cr} &= 128 + \tfrac{255-151.381}{1.402} = 128 + 73.91 = 201.91 \to 202. \end{aligned}$$

(b) No $(151, 43, 202)$:

$$\begin{aligned} R &= 151 + 1.402 \cdot 74 = 254.748 \to 255, \\ B &= 151 + 1.772 \cdot (-85) = 0.380 \to 0, \\ G &= (151 - 0.299 \cdot 254.748 - 0.114 \cdot 0.380)/0.587 = 74.787/0.587 = 127.41 \to 127. \end{aligned}$$

Iegūst $(255, 127, 0)$, nevis $(255, 128, 0)$. Pati matrica ir apgriežama; zudums rodas, noapaļojot Y, Cb, Cr līdz veseliem skaitļiem (un G aprēķinā kļūda pastiprinās, jo dala ar 0.587).

(c) $(R, G, B) = (1, 0.502, 0)$:

$$\begin{aligned} Y &= 0.299 + 0.587 \cdot 0.502 = 0.594, \\ I &= 0.5959 - 0.2746 \cdot 0.502 = 0.458, \\ Q &= 0.2115 - 0.5227 \cdot 0.502 = -0.051. \end{aligned}$$

$Q \approx 0$: oranžā krāsa atrodas gandrīz tieši uz I ass (pozitīvajā pusē). Tas atbilst lekcijas apgalvojumam, ka I apraksta pāreju no oranžās uz zilo krāsu, bet Q – no zaļās uz violeto.

(d) $(Y, -I, -Q) = (0.594, -0.458, 0.051)$, un M^{-1} dod

$$\begin{aligned} R &= 0.594 - 0.956 \cdot 0.458 + 0.621 \cdot 0.051 = 0.187, \\ G &= 0.594 + 0.272 \cdot 0.458 - 0.647 \cdot 0.051 = 0.685, \\ B &= 0.594 + 1.107 \cdot 0.458 + 1.704 \cdot 0.051 = 1.187. \end{aligned}$$

Tas ir zilgani zaļgans tonis, bet $B > 1$: krāsa ir ārpus RGB kuba ($\approx (48, 175, 303)$, ja reizina ar 255). YIQ telpas paralēlskaldnis ir lielāks par lineāro RGB kuba attēlu: RGB kubs ir tikai daļa no visiem iespējamajiem (Y, I, Q). Spilgti zila krāsa ar tik lielu gaišumu ($Y = 0.594$) nav iespējama, jo zilajam kanālam ir mazs svars (0.114) gaišumā. Atkodētājs šādu vērtību apgriež līdz $B = 1$, un tad mainās arī Y.

(e) Ja $R = G = B = v$, tad $Y = (0.299 + 0.587 + 0.114)v = v$, tātad $B - Y = R - Y = 0$ un $\mathrm{Cb} = \mathrm{Cr} = 128$. Matricas M otrajā un trešajā rindā koeficientu summa ir $0.5959 - 0.2746 - 0.3213 = 0$ un $0.2115 - 0.5227 + 0.3112 = 0$, tāpēc $I = Q = 0$.

(f) Y plaknē ir 10^6 baiti.

Izretināšana	Cb un Cr izmērs	Kopā
4:4:4	$2 \cdot 10^6$	$3 \cdot 10^6$ baiti (24 biti uz pikseli)
4:2:2	$2 \cdot 500 \cdot 1000 = 10^6$	$2 \cdot 10^6$ baiti (16 biti uz pikseli)
4:2:0	$2 \cdot 500 \cdot 500 = 5 \cdot 10^5$	$1.5 \cdot 10^6$ baiti (12 biti uz pikseli)

Redze daudz precīzāk uztver gaišuma detaļas nekā krāsu detaļas (sk. YIQ komponenšu attēlus lekcijā): kontūras un faktūra ir Y plaknē, bet Cb, Cr plaknes ir gludas. Tāpēc krāsainības izšķirtspējas samazināšana gandrīz nav redzama, bet ietaupa pusi datu.

3. uzdevums (Atkārtojiet bojājumu)

(a) un **(b)** Atbildes:

Attēls	Parametri	Redzamais bojājums	Mainītais solis
A	`--block-size 32 --quality 10`	rupjš kvadrātu režģis, gludas debesis “flīzēs”, viļņošanās gar kalnu malām izplatās tālu	3. (bloki) un 5. (kvantizācija)
B	`--keep-coefficients 3 --quality 90`	viss attēls izplūdis vienādi; sīkas detaļas (dēļi, sēta) pazūd, bloku iekšpusē gludas pārejas, viegli redzams 8×8 režģis	4. (koeficientu atlase)
C	`--color-space rgb`	krāsainas (sarkanas un zilganzaļas) apmales gar kontrastainām malām, izplūdis zaļais kanāls	1. (krāsu telpa)

Pārbaudes programma:

```python
import sys
import numpy as np
from PIL import Image

a = np.asarray(Image.open(sys.argv[1]).convert("RGB"), dtype=int)
b = np.asarray(Image.open(sys.argv[2]).convert("RGB"), dtype=int)
print("lielākā starpība:", np.abs(a - b).max())
```

```
python jpeg_lab.py barns_grand_tetons.png a.png --block-size 32 --quality 10
python compare.py a.png target-a.png        # lielākā starpība: 0
```

(c) Režģa solis ir 32 pikseļi, tātad `--block-size 32`. Bloku iekšpusē paliek tikai DC un daži zemo frekvenču koeficienti (gludi slīpumi, debesis vienā blokā gandrīz vienkrāsainas), tātad kvantizācija ir rupja: pazemināta

`--quality`. Precīzo vērtību 10 atrod, salīdzinot PSNR un izmēru ar dažiem palaidieniem (piemēram, $5, 10, 20$) un pēc tam pārbaudot sakritību.

(d) Ar `--color-space rgb` plaknes G un B apstrādā kā krāsainību: tās izretina 4:2:0 un kvantizē ar rupjāko krāsainības tabulu. Bet G plaknē ir gandrīz tikpat daudz detaļu kā R plaknē (un G dod 58.7% no gaišuma), tāpēc:

- kvalitāte ir sliktāka: izretināšana un rupjā kvantizācija iznīcina G un B detaļas, kuras acs labi redz (PSNR 20.56 dB, nevis 26.84 dB);
- izmērs ir lielāks (145.6 KiB, nevis 123.3 KiB): G un B plaknēm ir daudz lielu AC koeficientu, kurus nevar atmest, bet YCbCr gadījumā Cb un Cr plaknes ir gludas un saspiežas ļoti labi. YCbCr pārveidojums novērš korelāciju starp R, G, B; bez tā viena un tā pati informācija (malas) jākodē trijās plaknēs.

Ap tumšajām sētas latām parādās sarkanas/rozā un zilganzaļas apmales. R plakne (kodēta kā gaišums) paliek asa, bet G un B ir izplūdušas: tumšas latas malā R strauji samazinās, bet G un B – pakāpeniski. Tāpēc vienā pusē ir pārāk daudz G un B (zilganzaļš), otrā – pārāk maz (sarkanīgs).

(e) Parametri: `--block-size 16 --quality 5`. Kāpņu pakāpiena platums ir 16 pikseļi, tātad bloka izmērs ir 16. Pie kvalitātes 5 kvantizācijas soļi ir ļoti lieli (tabula pareizināta ar $5000/5/100 = 10$), tāpēc gludā gradienta blokā paliek tikai DC koeficients (vidējā vērtība), bet lēzenā slīpuma koeficients (pirmā AC frekvence) noapaļojas uz 0. Katrs bloks kļūst vienkrāsains, un gradients pārvēršas kāpnēs, kuru pakāpiens ir bloka platums. Plānās līnijas pēdējā joslā pazūd (paliek pelēks fons), jo to enerģija ir augstajās frekvencēs.

(f) Piemēra atbilde: 1 pikseli plats sarkans teksts uz zila fona. Abām krāsām ir līdzīgs gaišums, tāpēc gandrīz visa informācija ir krāsainības plaknēs, kuras 4:2:0 izretina, – teksts izplūst un kļūst violets, pat ja pazemina kvantizāciju (`--quality 95`). Bojājums pazūd ar `--chroma-subsampling 4:4:4`. Citi "neērti" attēli: šaha galdiņš ar 1 pikseļa rūtiņām (visa enerģija augstākajā frekvencē), asas malas, kas nesakrīt ar bloku robežām, lēzeni gradienti zemā kvalitātē (kāpnes, sk. (e) punktu).

Citi attēlu, audio un video kodējumi

4. uzdevums (Animēts attēls)

(a) Pulksteņa rādītājs, kas 50 kadros pagriežas par $360°$: kadrā i leņķis ir $\varphi_i = 360° \cdot i/50$. Animācija ir bezšuvju cilpa, ja kustība ir periodiska un perioda garums ir vesels kadru skaits: kadrs ar numuru 50 (kura vairs nav) sakristu ar kadru 0. Tāpēc ātrums jāizvēlas kā $360°/50$ kadrā (vai slīdošai joslai – ceļa garums, kas dalās ar kadru skaitu), un pēdējais kadrs nedrīkst atkārtot pirmo.

```python
import io
import numpy as np
from PIL import Image

W, H, FPS, N = 320, 180, 25, 50
BG, HAND = np.array([240, 236, 224]), np.array([200, 40, 40])

def draw(i, noise=None):
    phi = 2 * np.pi * i / N                          # kadrs N sakristu ar kadru 0
    yy, xx = np.mgrid[0:H, 0:W].astype(float)
    cx, cy, L = W / 2, H / 2, 80
    ex, ey = np.sin(phi), -np.cos(phi)               # rādītāja virziens
    t = np.clip((xx - cx) * ex + (yy - cy) * ey, 0, L)
    d = np.hypot(xx - cx - t * ex, yy - cy - t * ey)  # attālums līdz nogrieznim
    a = np.clip(4 - d, 0, 1)[..., None]              # nogludinātas malas
```

```
    img = (1 - a) * BG + a * HAND
    if noise is not None:
        img = img + noise
    return Image.fromarray(np.clip(np.round(img), 0, 255).astype(np.uint8), "RGB")

def sizes(frames):
    out = {}
    for fmt, kw in [("GIF", {}), ("WEBP", {"quality": 80}), ("AVIF", {"quality":
     ↪  80})]:
        buf = io.BytesIO()
        frames[0].save(buf, format=fmt, save_all=True, append_images=frames[1:],
                       duration=1000 // FPS, loop=0, **kw)
        out[fmt] = buf.tell()
    return out

rng = np.random.default_rng(1)
print(sizes([draw(i) for i in range(N)]))                                    #
 ↪  (b)
print(sizes([draw(i, rng.integers(-8, 9, (H, W, 1))) for i in range(N)]))    #
 ↪  (d)
static = rng.integers(-8, 9, (H, W, 1))
print(sizes([draw(i, static) for i in range(N)]))                            #
 ↪  (e)
```

(b), **(d)**, **(e)** Rezultāti (nesaspiesti kadri: $50 \cdot 320 \cdot 180 \cdot 3 = 8\,640\,000$ baiti):

Kadri	GIF	WebP	AVIF
tīrs fons	56.9 KB ($152:1$)	40.7 KB ($212:1$)	12.5 KB ($691:1$)
jauns troksnis katrā kadrā	1968 KB	329 KB	474 KB
viens trokšņa raksts visiem kadriem	130 KB	113 KB	51 KB

Ar tīru fonu AVIF ir vismazākais: tas kodē kadru starpības ar kustības kompensāciju un aritmētisko kodu. GIF katru kadru saspiež ar LZW (Pillow saglabā tikai mainīto taisnstūri ap rādītāju), WebP izmanto VP8 intra kodēšanu un kadru starpības.

(c) GIF kadrā ir ne vairāk kā 256 krāsas (palete). Gludā krāsu gradientā blakus esošas krāsas tiek attēlotas uz vienu paletes krāsu, un rodas joslas (*banding*). Ar `Dither.NONE` katru pikseli aizstāj ar tuvāko paletes krāsu – redzamas asas joslas. Ar `Dither.FLOYDSTEINBERG` noapaļošanas kļūdu izkliedē uz kaimiņu pikseļiem (*dithering*): vidēji pareizā krāsa iegūta ar divu paletes krāsu "rakstu". Joslas pazūd, bet parādās smalks graudains raksts, kas turklāt katrā kadrā var mainīties un palielina failu.

(d) Neatkarīgs troksnis katrā pikselī un kadrā ir neparedzams: intra prognoze no kaimiņu pikseļiem kļūdās par trokšņa lielumu, un arī iepriekšējais kadrs nepalīdz, jo tajā ir cits troksnis – P-freima atlikums ir troksnis ar divkāršu dispersiju. Troksnim ir maksimāla entropija, tāpēc GIF (tā LZW kods ir bezzudumu) jākodē gandrīz katrs bits (izmērs pieaug 35 reizes). Zudumradošie kodeki daļu trokšņa noapaļo, bet arī to izmēri pieaug 8–38 reizes.

(e) Ja troksnis visos kadros ir vienāds, tas ir daļa no fona: pirmajā kadrā tas ir jāiekodē (tāpēc izmērs ir lielāks nekā ar tīru fonu), bet nākamajos kadros fons atkārtojas precīzi, un jākodē tikai rādītāja kustība. Visvairāk iegūst

GIF ($1968 \to 130$ KB), jo tas ir bezzudumu formāts un iepriekš katrā kadrā kodēja visu jauno troksni; tagad mainās tikai neliels taisnstūris. AVIF joprojām ir mazākais, jo prognoze no iepriekšējā kadra precīzi atkārto fonu.

5. uzdevums (Skaņa frekvencēs)

```
import numpy as np
from scipy.fft import dct, idct
from scipy.io import wavfile

fs = 48000
t = np.arange(2 * fs) / fs
x = sum(0.2 * np.sin(2 * np.pi * f * t) for f in [440, 440 * 2 ** (4 / 12), 440 * 2
 ↪  ** (7 / 12)])
x += 0.002 * np.sin(2 * np.pi * 470 * t)
wavfile.write("chord.wav", fs, np.round(x * 32767).astype(np.int16))

X = np.abs(np.fft.rfft(x))
freqs = np.fft.rfftfreq(len(x), 1 / fs)
db = 20 * np.log10(X / X.max() + 1e-12)          # zīmēt db pret freqs (0..2000 Hz)

def compress(x, N, K):
    y = np.zeros_like(x)
    for s in range(0, len(x), N):
        c = dct(x[s:s + N], norm="ortho")
        c[np.argsort(np.abs(c))[:-K]] = 0         # patur K lielākos
        y[s:s + N] = idct(c, norm="ortho")
    return y

def snr(x, y):
    return 10 * np.log10(np.sum(x ** 2) / np.sum((x - y) ** 2))
```

(a) Virsotnes ir pie 440 Hz (0 dB), 470 Hz (-40.6 dB, jo $20\log_{10}(0.002/0.2) = -40$ dB), 554.5 Hz un 659.5 Hz. (C♯5 $= 554.37$ Hz un E5 $= 659.26$ Hz neatrodas tieši uz FFT frekvencēm, tāpēc virsotne ir tuvākajā frekvencē un nedaudz zemāka: -1.1 un -3.7 dB – spektra noplūde, *spectral leakage*.) FFT frekvenču izšķirtspēja ir $f_s/L = 1/T$: 2 sekunžu signālam 0.5 Hz, 0.02 sekunžu fragmentam (960 paraugi) 50 Hz. 2 sekunžu spektrā 440 Hz un 470 Hz ir 60 frekvenču attālumā, un tās ir labi atšķiramas; 0.02 s fragmentā abas nonāk blakus frekvencēs (450 un 500 Hz), un klusais tonis pazūd skaļā toņa “sānu daivās”. Īsi logi dod labu laika izšķirtspēju, bet sliktu frekvenču izšķirtspēju.

(b) Rezultāti ($N = 1024$; 470 Hz komponente kļūdā $x - x'$ izmērīta ar Hanna logu):

K	SNR	Kļūdas amplitūda pie 470 Hz
8	11.4 dB	0.0024
16	16.9 dB	0.0006
32	24.8 dB	0.0003
64	36.0 dB	0.00002

Ar $K = 8$ kļūda pie 470 Hz ir lielāka par paša toņa amplitūdu 0.002 – klusais tonis faktiski ir pazudis (aizstāts ar kropļojumiem). Ar $K \geq 32$ tas ir saglabāts ar labu precizitāti. Kadrā ar $N = 1024$ DCT frekvenču solis ir

$f_s/(2N) = 23.4$ Hz, tāpēc 440 Hz un 470 Hz atšķiras tikai par apmēram 1.3 koeficientiem: klusais tonis nav "atsevišķs" koeficients, bet maina skaļā toņa koeficientus. Pat oriģinālā 470 Hz tonis ir praktiski nedzirdams: tas ir 40 dB klusāks par 440 Hz toni tajā pašā kritiskajā joslā (ap 500 Hz josla ir apmēram 100 Hz plata), tātad zem maskēšanas sliekšņa. Kodētājs ar psihoakustisko modeli tam bitus netērētu vispār.

(c) Klikšķis 1.0 s ir paraugs $48\,000$. Ar $N = 4096$ tas atrodas kadrā, kas sākas paraugā $45\,056$, t.i., 61.3 ms pirms klikšķa; ar $N = 256$ – paraugā $47\,872$, t.i., 2.7 ms pirms klikšķa. Kļūda, ko rada klikšķis, izplūst pa *visu* kadru, tātad sākas kadra sākumā: 61.3 ms un 2.7 ms pirms klikšķa (sliktākajā gadījumā – veselu kadru: 85 ms un 5.3 ms). 61 ms ir daudz vairāk par priekšmaskēšanas ilgumu (5–20 ms), tāpēc garajā kadrā pirmsatbalss ir dzirdama kā "švīkstoņa" pirms klikšķa; īsajā kadrā tā ir priekšmaskēšanas robežās. Tāpēc MP3 un Opus pārejās pārslēdzas uz īsiem logiem, bet stacionāram signālam izmanto garus logus labākas frekvenču izšķirtspējas dēļ.

```
xc = x.copy(); xc[48000] += 0.9
for N in [4096, 256]:
    d = np.abs(compress(xc, N, N // 16) - compress(x, N, N // 16))   # klikšķa
 ↪  radītā izmaiņa
    first = np.nonzero(d[:48000] > 0.01)[0][0]
    print(N, (48000 - first) / fs * 1000, "ms")                      # 61.3 ms, 2.7
     ↪  ms
```

(d) Dzirdams 3000 Hz tonis (FFT virsotne pie 3000 Hz). Paraugu ņemšanas frekvencei 8000 Hz Naikvista frekvence ir 4000 Hz; 5000 Hz tonis ir virs tās, un tā paraugi sakrīt ar $8000 - 5000 = 3000$ Hz toņa paraugiem: $\sin(2\pi \cdot 5000 \cdot \frac{n}{8000}) = \sin(2\pi n - 2\pi \cdot 3000 \cdot \frac{n}{8000}) = -\sin(2\pi \cdot 3000 \cdot \frac{n}{8000})$ (*aliasing*).

(e) Ar $q_k = q_0(1 + k/64)$ un $q_0 = 0.05$ katrā kadrā vidēji ir 49 nenulles koeficienti un $\mathrm{SNR} = 30.7$ dB; ar $q_0 = 0.02$ – 68 koeficienti un 35.9 dB. Salīdzinājumam "K lielākie" ar $K = 48$ dod 31.6 dB, ar $K = 68$ – 36.9 dB. SNR ziņā kvantizācija ir nedaudz sliktāka (jo "K lielākie" koeficientus nekvantizē, bet tad jānosūta arī to pozīcijas un precīzas vērtības!), bet tā ir reālistiskāka: kvantizētas vērtības kodē ar maz bitiem, un augsto frekvenču rupjāka kvantizācija atbilst dzirdes jutībai.

6. uzdevums (Teorijas jautājumi par kodekiem)

- **(a)** *Aplams.* Informāciju zaudē arī krāsainības izretināšana (2. solis) un noapaļošana pēc krāsu telpas maiņas (sk. 2. uzdevuma (b) punktu). Bezzudumu ir tikai paša DCT matemātiskā definīcija; vesela skaitļa aritmētikā arī tā ievieš nelielas noapaļošanas kļūdas.
- **(b)** *Aplams.* Formula $Q^2/12$ pieņem, ka noapaļošanas kļūda ir vienmērīgi sadalīta intervālā $[-Q/2; Q/2]$. Tas ir aptuveni pareizi lieliem koeficientiem. Koeficientam, kas noapaļojas uz 0, kļūda ir pats koeficients y_k, un tipiski augsto frekvenču koeficienti ir daudz mazāki par $Q/2$, tāpēc kļūda ir daudz mazāka par $Q^2/12$. (Turklāt JPEG katram koeficientam ir savs $Q_{u,v}$.)
- **(c)** *Aplams.* Bitu skaits nenulles koeficientam aug apmēram kā $\log_2(|y|/Q)$, tāpēc divkāršots solis ietaupa apmēram vienu bitu uz koeficientu un pārvērš daļu koeficientu par nullēm. Piemēram, `jpeg_lab.py` attēlam `barns_grand_tetons.png`: `--quality 50` dod 123.3 KiB, bet `--quality 25` (visas tabulas divreiz lielākas) – 75.0 KiB, t.i., 1.64 reizes mazāk.
- **(d)** *Patiess.* $\mathrm{PSNR} = 10 \log_{10} \frac{255^2}{\mathrm{MSE}}$; starpība 20 dB nozīmē, ka MSE attiecība ir $10^{20/10} = 100$.
- **(e)** *Aplams.* Iekodētājs meklē labāko bloku sadalījumu, prognozes režīmu, transformāciju un filtru parametrus (*rate–distortion optimization*), bet atkodētājs tos nolasa no faila un izpilda tikai vienu variantu. Atkodēšana ir daudz ātrāka.
- **(f)** *Aplams.* P-freimi atsaucas uz iepriekšējiem I vai P freimiem, B-freimi – uz enkurfreimiem abās pusēs. Skatīšanos var sākt tikai no I-freima (atslēgas freima); citādi vispirms jāatkodē visa ķēde no iepriekšējā I-freima.

- **(g)** *Aplams.* Opus (CELT) katrai joslai precīzi kodē tikai enerģiju; zemā bitu ātrumā augsto joslu formu aizpilda ar "salocītu" zemāku frekvenču spektru. Frekvences līdz 20 kHz ir *klātesošas* ar pareizu skaļumu, bet to precīzās vērtības nav saglabātas.
- **(h)** *Patiess.* WebM ir Matroska konteinera apakškopa (ierobežota kodeku izvēle), tāpēc `.webm` fails jau ir derīgs Matroska fails. Paplašinājuma maiņa neko faila iekšienē nemaina, un atskaņotāji, kas atbalsta MKV un VP9/Opus, to atskaņos. (Pretējais virziens – `.mkv` uz `.webm` – der tikai tad, ja failā ir WebM atļautie kodeki.)

Heminga kodi un Tanera grafi

7. uzdevums (Heminga kods un tā paplašinājums)

(a) $x_1x_2x_3x_4 = 1011$: $y_1 = 1 \oplus 0 \oplus 1 = 0$, $y_2 = 1 \oplus 0 \oplus 1 = 0$, $y_3 = 1 \oplus 1 \oplus 1 = 1$. Koda vārds: $x_1 x_2 x_3 y_1 x_4 y_2 y_3 = $ `1010101`.

(b) Saņemts `1110101`: $x_1 = 1, x_2 = 1, x_3 = 1, y_1 = 0, x_4 = 1, y_2 = 0, y_3 = 1$.

- $x_1 \oplus x_2 \oplus x_3 = 1 \neq y_1$, tātad $s_1 = 1$;
- $x_1 \oplus x_2 \oplus x_4 = 1 \neq y_2$, tātad $s_2 = 1$;
- $x_1 \oplus x_3 \oplus x_4 = 1 = y_3$, tātad $s_3 = 0$.

Sindroms $s_1s_2s_3 = 110$ ir pozīcijas x_2 binārais indekss. Vispārīgi: bits ar indeksu $i_1i_2i_3$ ietilpst pārbaudē y_j (kuras indeksā vieninieks ir j-tajā vietā) tad un tikai tad, ja $i_j = 1$. Tāpēc kļūda šajā bitā maina tieši tās pārbaudes, kurām $i_j = 1$, un sindroms ir $i_1i_2i_3$. Izlabotais vārds ir `1010101`, ziņojums `1011`.

(c) Saņemts `1010110`: $x_1x_2x_3 = 101$, $y_1 = 0$, $x_4 = 1$, $y_2 = 1$, $y_3 = 0$. Pārbaudes: $s_1 = (1 \oplus 0 \oplus 1) \oplus 0 = 0$, $s_2 = (1 \oplus 0 \oplus 1) \oplus 1 = 1$, $s_3 = (1 \oplus 1 \oplus 1) \oplus 0 = 1$. Sindroms 011 norāda uz x_4. Atkodētājs pārslēdz x_4 un iegūst `1010010` – derīgu, bet nepareizu koda vārdu, un izvada ziņojumu `1010` (nevis `1011`). Divas kļūdas ar šo kodu nevar pat pamanīt.

(d) Divi dažādi Heminga koda vārdi atšķiras vismaz 3 pozīcijās. Ja tie atšķiras tieši 3 (vai jebkurā nepāra skaitā) pozīcijās, tad to vieninieku skaitu paritāte ir dažāda, tātad atšķiras arī paritātes biti y_0, un kopā ir vismaz 4 atšķirības. Ja tie atšķiras pāra skaitā pozīciju, tad jau vismaz 4.

Atkodētājs aprēķina sindromu $S = s_1s_2s_3$ un visu 8 bitu paritāti P (pareizam vārdam $P = 0$):

S	P	Secinājums
000	0	kļūdu nav
$\neq 000$	1	viena kļūda pozīcijā S – izlabo
000	1	viena kļūda pašā bitā y_0
$\neq 000$	0	divas kļūdas – pamana, bet nelabo

Viena kļūda vienmēr maina P, divas – nemaina. (c) punktā: pārraidīts `1010101` ar $y_0 = 0$ (četri vienīnieki). Saņemts `1010110` ar $y_0 = 0$: vieninieku skaits ir 4, $P = 0$, bet $S = 011 \neq 000$ – atkodētājs ziņo par divām kļūdām, nevis izvada nepareizu ziņojumu.

(e) $y_0 = x_1 \oplus x_2 \oplus x_3 \oplus x_4 \oplus y_1 \oplus y_2 \oplus y_3$. Ievietojot y_1, y_2, y_3, x_1 ir 4 reizes, bet x_2, x_3, x_4 – katrs 3 reizes. Pēc moduļa 2 paliek $y_0 = x_2 \oplus x_3 \oplus x_4$.

8. uzdevums (Tanera grafs un dzēsumi)

(a)

$$H = \begin{pmatrix} 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

Tanera grafā ir 8 bitu mezgli un 4 pārbaužu mezgli; $b_1, \dots, b_4$ katrs ir savienots ar 3 pārbaudēm, $b_5, \dots, b_8$ – katrs ar vienu (b_{4+i} ar p_i). Katrā pārbaudē p_i ir tieši viens no bitiem $b_5, \dots, b_8$, tāpēc to var izteikt: $b_5 = b_1 \oplus b_2 \oplus b_3$, $b_6 = b_1 \oplus b_2 \oplus b_4$, $b_7 = b_1 \oplus b_3 \oplus b_4$, $b_8 = b_2 \oplus b_3 \oplus b_4$. Tātad $b_1, \dots, b_4$ ir brīvi, un kodā ir $2^4 = 16$ vārdi. `1011` dod $b_5 = 0, b_6 = 0, b_7 = 1, b_8 = 0$, tātad `10110010`.

Tas ir 7. uzdevuma paplašinātais Heminga kods: b_5, b_6, b_7 ir y_1, y_2, y_3, bet $b_8 = x_2 \oplus x_3 \oplus x_4 = y_0$ (7. uzdevuma (e) punkts). Atšķiras tikai bitu secība.

(b) Zināms $b_3 = 1, b_4 = 1, b_6 = 0, b_7 = 1, b_8 = 0$.

1. p_3 (b_1, b_3, b_4, b_7) satur tikai vienu nezināmo: $b_1 = b_3 \oplus b_4 \oplus b_7 = 1$.
2. p_4 (b_2, b_3, b_4, b_8): $b_2 = 1 \oplus 1 \oplus 0 = 0$.
3. p_1 (b_1, b_2, b_3, b_5): $b_5 = 1 \oplus 0 \oplus 1 = 0$.

Pārbaude ar p_2: $b_1 \oplus b_2 \oplus b_4 \oplus b_6 = 1 \oplus 0 \oplus 1 \oplus 0 = 0$. (Sākumā p_1 un p_2 satur divus nezināmos, tāpēc jāsāk ar p_3 vai p_4.)

(c) Katrā pārbaudē ir trīs no bitiem $b_1, \dots, b_4$, tātad trīs nezināmie – nevienu pārbaudi nevar izmantot, un noplēšana apstājas uzreiz. (Tādu dzēsumu kopu, kurā katra pārbaude satur vai nu nulli, vai vismaz divus dzēstos bitus, sauc par *apstāšanās kopu*, *stopping set*.) Tomēr sistēma

$$\begin{aligned} b_1 \oplus b_2 \oplus b_3 &= b_5 = 0, \\ b_1 \oplus b_2 \oplus b_4 &= b_6 = 0, \\ b_1 \oplus b_3 \oplus b_4 &= b_7 = 1, \\ b_2 \oplus b_3 \oplus b_4 &= b_8 = 0 \end{aligned}$$

ir viennozīmīgi atrisināma. Saskaitot visus vienādojumus, katrs b_i ir trīs reizes: $b_1 \oplus b_2 \oplus b_3 \oplus b_4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$. Pieskaitot to katram vienādojumam: $b_4 = 1 \oplus 0 = 1, b_3 = 1 \oplus 0 = 1, b_2 = 1 \oplus 1 = 0, b_1 = 1 \oplus 0 = 1$. Atjaunots `1011` – Gausa metode (vai maksimālās ticamības atkodēšana) var vairāk nekā noplēšana.

(d) Ja divi koda vārdi sakrīt visās neizdzēstajās pozīcijās, atkodētājs tos nevar atšķirt. To starpība (XOR) ir koda vārds, kura vienīnieki ir tikai dzēstajās pozīcijās. Kods ir lineārs un tā minimālais attālums ir 4, tāpēc nenulles koda vārdā ir vismaz 4 vieninieki: jebkurus 3 dzēsumus var atjaunot. Četru dzēsumu kopa nav atjaunojama tad un tikai tad, ja tā ir kāda koda vārda ar četriem vieniniekiem pozīciju kopa. Piemēram, ziņojums `1000` dod `10001110`, tātad dzēsumus $\{b_1, b_5, b_6, b_7\}$ nevar atjaunot: nosūtīto vārdu un vārdu, kam pieskaitīts `10001110`, nevar atšķirt.

(e)

```
import numpy as np

def peel(checks, received):
```

```
    """checks: pārbaužu saraksts (bitu indeksu saraksti); received: 0, 1 vai
     ↪  None."""
    b = list(received)
    progress = True
    while progress and None in b:
        progress = False
        for chk in checks:
            unknown = [j for j in chk if b[j] is None]
            if len(unknown) == 1:
                b[unknown[0]] = sum(b[j] for j in chk if b[j] is not None) % 2
                progress = True
    return b

def random_ldpc(n, dv=3, dc=6, rng=None):
    """(3, 6)-regulārs Tanera grafs: n*dv šķautņu "galus" sajauc nejaušā secībā."""
    sockets = rng.permutation(np.repeat(np.arange(n), dv))
    return [list(sockets[i * dc:(i + 1) * dc]) for i in range(n * dv // dc)]

rng = np.random.default_rng(2026)
n = 1200
checks = random_ldpc(n, rng=rng)
for eps in [0.30, 0.35, 0.40, 0.45, 0.50]:
    ok = sum(None not in peel(checks, [None if rng.random() < eps else 0 for _ in
     ↪  range(n)])
             for _ in range(100))
    print(eps, ok)
```

Rezultāti (veiksmīgi atkodēto reižu skaits no 100):

ε	0.30	0.35	0.40	0.42	0.45	0.50
veiksmīgi	100	100	92	57	8	0

Pietiek pārbaudīt nulles vārdu: noplēšanas algoritma gaita (kuras pārbaudes kurā brīdī satur vienu nezināmo) ir atkarīga tikai no dzēsumu pozīcijām, nevis no bitu vērtībām. Ja koda vārds ir c, tad visas atjaunotās vērtības ir c biti tieši tad, kad nulles vārdam tās ir nulles (kods ir lineārs: XOR ar c pārvērš vienu gadījumu otrā). Atkodēšana strauji pasliktinās ap $\varepsilon \approx 0.42$–0.43, tas atbilst teorētiskajai robežai 0.429. Garākam kodam pāreja būtu vēl asāka. (Nejaušajā grafā vienā pārbaudē var gadīties viens bits divreiz; tas ir reti un rezultātu būtiski neietekmē.)

Rīda-Solomona kodi un galīgie lauki

9. uzdevums (Rīda-Solomona kods laukā GF(7))

(a) $1 \cdot 1 = 1, 2 \cdot 4 = 8 \equiv 1, 3 \cdot 5 = 15 \equiv 1, 6 \cdot 6 = 36 \equiv 1$:

a	1	2	3	4	5	6
a^{-1}	1	4	5	2	3	6

(b) $f(x) = 3x + 5$: $f(0) = 5$, $f(1) = 8 \equiv 1$, $f(2) = 11 \equiv 4$, $f(3) = 14 \equiv 0$, $f(4) = 17 \equiv 3$. Nosūta $5, 1, 4, 0, 3$.

(c) Pietiek ar jebkurām $k = 2$ pareizām vērtībām, tāpēc var atjaunot $s - k = 3$ dzēsumus. Kļūdas: $c \le (s-k)/2 = 1.5$, tātad 1 kļūda. Argumentiem jābūt dažādiem lauka elementiem, tāpēc $s \le 7$; ar $s = 7$ var labot $(7-2)/2 = 2.5$, t.i., 2 kļūdas (vai 5 dzēsumus).

(d) Zināms $f(1) = 1$ un $f(4) = 3$. Lagranža bāzes polinomi:

$$L_1(x) = \frac{x-4}{1-4} = (x-4)\cdot(-3)^{-1}, \qquad L_4(x) = \frac{x-1}{4-1} = (x-1)\cdot 3^{-1}.$$

Pēc moduļa 7: $-3 \equiv 4$, $4^{-1} = 2$, tātad $L_1(x) = 2(x-4) = 2x-8 \equiv 2x+6$; $3^{-1} = 5$, tātad $L_4(x) = 5(x-1) = 5x-5 \equiv 5x+2$. Tad

$$f(x) = 1\cdot(2x+6) + 3\cdot(5x+2) = 17x + 12 \equiv 3x + 5.$$

Ziņojums ir $3, 5$ (pārbaude: $f(1) = 8 \equiv 1$, $f(4) = 17 \equiv 3$).

(e) $k = 2$, $c = 1$: $Y(x) = x + y_0$, $Z(x) = z_2x^2 + z_1x + z_0$, vienādojumi $Z(i) = (i + y_0)\cdot r_i$ pēc moduļa 7 ($3^2 = 9 \equiv 2$, $4^2 = 16 \equiv 2$):

$$\begin{aligned}
i=0: \quad & z_0 && \equiv 5y_0,\\
i=1: \quad & z_2+z_1+z_0 && \equiv 1+y_0,\\
i=2: \quad & 4z_2+2z_1+z_0 && \equiv 6(2+y_0) \equiv 5+6y_0,\\
i=3: \quad & 2z_2+3z_1+z_0 && \equiv 0,\\
i=4: \quad & 2z_2+4z_1+z_0 && \equiv 3(4+y_0) \equiv 5+3y_0.
\end{aligned}$$

- Atņemot $i = 3$ vienādojumu no $i = 4$ vienādojuma: $z_1 \equiv 5 + 3y_0$.
- Ievietojot $z_0 = 5y_0$ un z_1 vienādojumā $i = 3$: $2z_2 + 15 + 9y_0 + 5y_0 = 2z_2 + 1 + 0\cdot y_0 \equiv 0$ (jo $14y_0 \equiv 0$), tātad $2z_2 \equiv 6$ un $z_2 \equiv 3$.
- Vienādojums $i = 1$: $3+5+3y_0+5y_0 = 8+8y_0 \equiv 1+y_0$ – izpildās jebkuram y_0 (nedod jaunu informāciju).
- Vienādojums $i = 2$: $12 + 10 + 6y_0 + 5y_0 = 22 + 11y_0 \equiv 1 + 4y_0 \equiv 5 + 6y_0$, tātad $-2y_0 \equiv 4$, $5y_0 \equiv 4$, $y_0 \equiv 4\cdot 5^{-1} = 4\cdot 3 = 12 \equiv 5$.
- Tad $z_1 \equiv 5 + 15 = 20 \equiv 6$ un $z_0 \equiv 25 \equiv 4$.

$Y(x) = x + 5$, tās sakne ir $x = 2$ ($2 + 5 = 7 \equiv 0$): kļūdaina ir vērtība r_2. $Z(x) = 3x^2 + 6x + 4$. Dalām ar $Y(x)$:

$$\begin{aligned}
3x^2+6x+4-3x(x+5) &= -9x+4 \;\equiv\; 5x+4,\\
5x+4-5(x+5) &= -21 \;\equiv\; 0.
\end{aligned}$$

$f(x) = 3x + 5$ bez atlikuma: ziņojums $3, 5$, un pareizā vērtība pozīcijā 2 ir $f(2) = 4$ (nevis 6).

(f) Taisne caur $(0, 5)$ un $(1, 1)$: $f(x) = 5 + (1-5)x \equiv 3x + 5$. Tā iet arī caur $(3, 0)$ un $(4, 3)$, bet ne caur $(2, 6)$ ($f(2) = 4$). Tātad četri punkti atrodas uz vienas taisnes, un tā ir vienīgā: ja divas taisnes ietu katra caur četriem no pieciem punktiem, tām būtu vismaz 3 kopīgi punkti, bet dažādām taisnēm ir ne vairāk kā viens kopīgs punkts. Pārlasē jāpārbauda visas $\binom{s}{k}$ punktu kopas; ja $s = 255$, $k = 223$, tas ir aptuveni 10^{40} kopas, bet Berlekampa-Velča metode prasa $O(s^3)$ darbību.

10. uzdevums (Lauks GF(8) un Rīda-Solomona kodi baitiem)

(a) Katru nākamo pakāpi iegūst, reizinot ar x un aizstājot $x^3 = x + 1$:

i	0	1	2	3	4	5	6	7
α^i	1	x	x^2	$x+1$	x^2+x	x^2+x+1	x^2+1	1
biti	001	010	100	011	110	111	101	001

Piemēram, $\alpha^5 = x \cdot (x^2+x) = x^3+x^2 = x^2+x+1$ un $\alpha^6 = x^3+x^2+x = x^2+1$. Starp $\alpha^0, \ldots, \alpha^6$ ir visas septiņas nenulles bitu virknes, tātad α ir primitīvs elements, un $\alpha^7 = 1$.

(b) $x^2+1 = \alpha^6$, $x+1 = \alpha^3$, tātad $(x^2+1)(x+1) = \alpha^9 = \alpha^2 = x^2$. $x^2+x = \alpha^4$, tātad $(x^2+x)^{-1} = \alpha^{-4} = \alpha^3 = x+1$. Lekcijas tabulā: rinda x^2+1, kolonna $x+1$ – x^2; rinda x^2+x, kolonna $x+1$ – 1. Saskaitot polinomus, saskaita atbilstošos koeficientus pēc moduļa 2, un $0 \oplus 0 = 0, 0 \oplus 1 = 1, 1 \oplus 1 = 0$ – tas ir XOR katram bitam (pārnesumu nav).

(c) $f(x) = \alpha x + 1$:

- \$f(0) = 1 = \$ 001;
- \$f(1) = 𝛼 + 1 = \$ 011;
- \$f(𝛼) = 𝛼^2 + 1 = \$ 101;
- \$f(𝛼^2) = 𝛼^3 + 1 = (x + 1) + 1 = x = \$ 010.

Ziņojumā ir $2 \cdot 3 = 6$ biti, koda vārdā $4 \cdot 3 = 12$ biti. Kods labo $(4-2)/2 = 1$ kļūdainu simbolu (līdz 3 bitiem viena simbola robežās).

(d)

```
import numpy as np
import galois

GF8 = galois.GF(2**3)                       # p(x) = x^3 + x + 1
a = GF8.primitive_element                   # alpha = x = 2
print([format(int(a**i), "03b") for i in range(8)])
print(GF8(5) * GF8(3), GF8(6) ** -1)        # x^2 = 4, x + 1 = 3

def points(GF, s):
    """Argumenti 0, 1, alpha, alpha^2, ..., alpha^(s-2)."""
    al = GF.primitive_element
    return GF([0] + [int(al**i) for i in range(s - 1)])

def rs_encode(msg, s):
    return galois.Poly(msg)(points(type(msg), s))   # msg = a_1, ..., a_k

def berlekamp_welch(r, k):
    GF = type(r); s = len(r); c = (s - k) // 2
    xs = points(GF, s)
    # nezināmie z_0..z_{k+c-1}, y_0..y_{c-1}:  Z(x_i) - r_i (y_0 + ... ) = r_i
     ↪  x_i^c
    A = GF(np.zeros((s, k + 2 * c), dtype=int))
    for j in range(k + c):
        A[:, j] = xs ** j
    for j in range(c):
        A[:, k + c + j] = -r * xs ** j
    M = np.concatenate([A, (r * xs ** c)[:, None]], axis=1).row_reduce()
    sol = GF(np.zeros(k + 2 * c, dtype=int))
```

```
    for row in M:                                       # brīvie mainīgie = 0
        nz = np.nonzero(row[:-1])[0]
        if len(nz) == 0:
            if row[-1] != 0:
                return None                             # sistēmai nav atrisinājuma
            continue
        sol[nz[0]] = row[-1]
    Z = galois.Poly(sol[:k + c][::-1])
    Y = galois.Poly(np.concatenate([GF([1]), sol[k + c:][::-1]]))
    q, rem = divmod(Z, Y)
    return q if rem == 0 else None
```

Sistēma ir nenoteikta, ja kļūdu ir mazāk par c, tāpēc `np.linalg.solve` (kas prasa kvadrātisku, apgriežamu matricu) neder; `row_reduce` (Gausa metode) un brīvo mainīgo aizstāšana ar 0 dod vienu no atrisinājumiem, un pēc lekcijas apgalvojuma visi atrisinājumi dod vienu un to pašu Z/Y. Laukā GF(2^8) ar $k = 8$, $s = 16$ var labot $c = 4$ kļūdas. Ar 200 nejaušiem mēģinājumiem: 3 un 4 kļūdas izlabotas visos 200 gadījumos, 5 kļūdas – nevienā (sistēmai nav atrisinājuma vai dalījumam ir atlikums; retos gadījumos varētu iegūt arī citu, nepareizu koda vārdu).

(e)

```
rs = galois.ReedSolomon(255, 223)
text = "Algoritmi telekomunikacijas un drosibas risinajumos".encode()
m = rs.field(np.frombuffer(text.ljust(223, b" "), dtype=np.uint8))
c = rs.encode(m)
rng = np.random.default_rng(7)
for L in [16, 17]:
    e = c.copy()
    e[40:40 + L] ^= rs.field(rng.integers(1, 256, L))     # L sabojāti baiti pēc
↪   kārtas
    d, nerr = rs.decode(e, errors=True)
    print(L, nerr, bytes(np.array(d, dtype=np.uint8))[:len(text)] == text)
```

Ar 16 sabojātiem baitiem atkodētājs atgriež pareizu ziņojumu un izlaboto kļūdu skaitu 16; ar 17 – kļūdu skaitu -1 (atkodēšana neizdevās), un atgrieztie baiti nesakrīt ar oriģinālo ziņojumu. Kodā ir $255 - 223 = 32$ kontrolbaiti, tāpēc tas labo līdz 16 kļūdainiem baitiem. Ja kļūdu ir vairāk, atkodētājs parasti to pamana, bet var arī (ļoti reti) atrast nepareizu koda vārdu.

(f) 128 pēc kārtas sabojāti biti aizņem $\lceil 128/7 \rceil = 19$ vai 20 Heminga bloku. Pilnībā skartajos blokos ir vairākas kļūdas (gadījuma vērtībām vidēji pusē no 7 bitiem), kuras Heminga kods nevar izlabot – izlabot var, ja paveicas, tikai malējos blokus, kuros ir viena kļūda. Tātad praktiski visi 128 biti paliek kļūdaini (turklāt atkodētājs "izlabo" vēl citus bitus). Rīda-Solomona kodam simbols ir viss baits: 128 pēc kārtas sabojāti biti ir tikai 16 (vai 17, ja nesakrīt ar baitu robežām) kļūdaini simboli, un tie tiek izlaboti. Kļūdu uzliesmojumā ir svarīgs nevis kļūdaino bitu, bet kļūdaino *simbolu* skaits. Kompaktdiskos papildus izmanto *starpkodēšanu* (*interleaving*, CIRC): vienas koda grupas simbolus izkaisa pa garu diska posmu, tāpēc skrāpējums sabojā tikai dažus simbolus katrā no daudzām grupām.